

Flag : `picoCTF{caesar_d3cr9pt3d_78250af...}`

Résumé de la faille

Un fichier `enc_flag` contient une chaîne encodée en **base64, deux fois de suite** (imbriqué), qui révèle ensuite un flag chiffré en **César** (décalage de 7). Le hint "engaging in various decoding processes" (au pluriel) annonçait la nécessité d'enchaîner plusieurs décodages.

⚡ Méthodo — les étapes

- 1. Lire le hint attentivement - "Various decoding **processes**" (pluriel) → un seul décodage ne suffira probablement pas, il faut en enchaîner plusieurs.
- 2. Premier décodage (CyberChef, From Base64)

```
Input  : YidkM0JxZGtwQlRYdHFhR3g2YUhsZmF6TnFLVGwzWVR0clh6YzRNaLV3YUcxZWZRPt0nCG==
Output : b'd3BqdkpBTXtqaGx6aHLfazNqeTL3YTnrXzc4MjUwaG1qfQ=='
```

- Le résultat se termine encore par `==` et ressemble encore à du base64 → signe qu'il faut décoder une seconde fois.

⚠ Piège : le `b'...'` n'est PAS du contenu

Le `b'...'` autour du résultat est une notation d’AFFICHAGE de CyberChef (convention empruntée à Python signalant "ceci est du binaire/bytes"), **pas une partie du message décodé**. Il ne faut garder que ce qu'il y a **entre** les deux apostrophes pour la suite :

```
d3BqdkpBTXtqaGx6aHLfazNqeTL3YTnrXzc4MjUwaG1qfQ==
```

Copier le `b'` ou le `'` par erreur casse le décodage suivant.

Astuce pour éviter ce piège : empiler directement une 2e opération "From Base64" dans la même recette CyberChef, plutôt que copier-coller manuellement entre deux étapes séparées — CyberChef gère alors le passage de données en interne, sans jamais exposer le `b'...'`.

4. Deuxième décodage (From Base64 à nouveau)

```
Output : wpjvJAM{jhlzhy_k3jy9wa3k_78250hmj}
```

- Le résultat n'est plus du base64 : structure `{...}` intacte (accolades, underscores, chiffres inchangés), mais les LETTRES semblent aléatoires → signature d'un chiffrement par SUBSTITUTION simple (César/ROT/Vigenère), pas un nouvel encodage.

5. Reconnaître qu'il s'agit de César

Indices : - Structure et ponctuation intactes, seules les lettres changent - Un mot de même longueur que "picoCTF" apparaît juste avant `{ ( wpjvJAM vs picoCTF )` → suggère un décalage fixe de lettres - Contrairement à un chiffrement fort (AES...), le texte reste "presque lisible" dans sa structure

6. Casser le César avec dcode.fr (brute-force) - Aller sur **dcode.fr** → Chiffre de César - Coller le texte, cliquer "DECRYPT (BRUTEFORCE)" - L'outil teste automatiquement les 25 décalages possibles et les trie par probabilité (score de lisibilité) - Décalage **7** donne : `picoCTF{caesar_d3cr9pt3d_78250af...}` 🎯

🔍 Reconnaître une substitution simple (César/ROT/Vigenère)

Signes caractéristiques d'un texte chiffré par SUBSTITUTION (pas un encodage type base64/hex) : - La **ponctuation, les espaces, les chiffres restent identiques** - Seules les **lettres** sont remplacées par d'autres lettres - La **longueur du texte** est inchangée - Un mot attendu (comme "picoCTF") peut être "deviné" en comparant lettre par lettre avec le résultat obtenu

À l'inverse, un chiffrement fort (AES) ou un encodage binaire mal décodé donne du charabia complet, sans structure reconnaissable.

🧰 Outils utilisés

Outil	Rôle
CyberChef (From Base64, empilé x2)	Décoder les deux couches de base64
dcode.fr (Chiffre de César, brute-force)	Tester les 25 décalages automatiquement

✔ Réflexes à retenir

- Un hint au **pluriel** ("processes", "decodings"...) annonce souvent plusieurs étapes à enchaîner, pas une seule.
- Si un résultat décodé **ressemble encore** à l'encodage précédent (même alphabet, `=` de padding...) → réappliquer le même décodage.
- Le `b'...'` de CyberChef (ou de Python en général) est un marqueur d’AFFICHAGE, jamais du contenu réel — toujours extraire l'intérieur.
- Structure/ponctuation intacte + lettres changées = penser immédiatement à une substitution simple (César/ROT/Vigenère), pas à un encodage classique.
- dcode.fr fait un bruteforce automatique très efficace sur César (seulement 26 décalages possibles, donc rapide à tout tester).